

CHM MediaHub

Microservices Migration Plan

Monolith Decomposition, Codebase Migration,
AWS Cognito Identity (CHT-Owned IdP), and GoTrue Decommission

Plan Date: June 03, 2026

Audience: CHM engineering, CHT platform team, leadership

Scope: MediaHub backend, frontend, workers, and identity boundary

IDENTITY CORRECTION: CHT will own the IdP using AWS Cognito. This plan explicitly adopts Cognito as the long-term identity platform for CHT end users. MediaHub will NOT remain a shared user-facing auth hub. GoTrue will be decommissioned for all non-admin access.

1. Executive Summary

This plan describes how to migrate CHM MediaHub from a single monolithic FastAPI application on one EC2 instance to a decomposed microservices architecture on AWS, while simultaneously redefining the identity boundary so that CHT owns end-user authentication via AWS Cognito and MediaHub becomes an admin-only operations portal plus machine-to-machine API surface for CHT.

Today, MediaHub, CHT Platform, and the chatbot share a GoTrue (Supabase) auth stack hosted alongside MediaHub. End users (KOLs, HCPs, pharma clients) can obtain JWTs that grant access to MediaHub dashboards if they have a public.users row. This creates coupling, security sprawl, and operational risk. The target state separates concerns cleanly.

1.1 Target-State Summary

- CHT Platform owns AWS Cognito User Pool(s) for all end-user identity (KOL, HCP, pharma client, clinician).
- MediaHub UI is admin-only (CHM superadmin/admin/editor roles). No general CHT user login to MediaHub.
- CHT consumes MediaHub data via `/api/public/*` using X-API-Key (PUBLIC_API_KEY) - server-to-server, not user JWT passthrough.
- ops-console syncs branded content via WEBHOOK_API_KEY on `/webhook/sync` - unchanged pattern, hardened.
- GoTrue is decommissioned after admin accounts migrate to Cognito admin app client.
- Monolith splits into: core-api, worker, render-worker, report-worker, kb-worker, frontend - on ECS Fargate.
- Shared data layer: RDS PostgreSQL, ElastiCache Redis, S3 - not co-located on app EC2.

1.2 Guiding Principles

1. Strangler fig migration - extract services incrementally; never big-bang rewrite.
2. CHT owns Cognito; MediaHub validates Cognito JWTs only for admin UI sessions.
3. API keys for machine clients; Cognito for human admin sessions; no hybrid user auth on public API.

4. Same PostgreSQL schema initially; split databases only when service boundaries are proven.
5. Python/FastAPI retained for domain services - no language rewrite.

2. Current State (Monolith + Shared GoTrue)

2.1 Monolith Composition

Single FastAPI process (backend/main.py) serves all HTTP routes and runs APScheduler cron jobs in-process. GPU video rendering, LLM report generation, platform sync, HCP intel polling, and KB correction all share one container with NVIDIA runtime.

- HTTP routers: analytics, auth, webhook, public_api, reports, render, conversations, hcp_intel, knowledge_base, ...
- Background jobs: YouTube/LinkedIn/X/FB/IG sync, LinkedIn stats refresh, HCP intel polling, AI summaries, tag pipelines
- Long-running: report PPTX generation, FFmpeg renders, KB LLM correction (blocking or BackgroundTasks)
- Frontend: Next.js dashboard (all roles including pharma viewers)
- Data: PostgreSQL + Redis in Docker on same EC2; files on local disk
- Chatbot: separate bare-metal service on same EC2 (ChromaDB + OpenAI)

2.2 Current Identity Model (Problem)

- GoTrue container (gotrue service) in docker-compose.prod.yml - shared JWT secret with backend
- GOTRUE_URI_ALLOW_LIST includes mediahub, chmbot, testapp, cht-platform-tool-demo.vercel.app
- AuthService decodes GoTrue JWT OR legacy MediaHub JWT (auth_service.py)
- get_current_user requires public.users row - CHT-only users without row get 403, but invited users get full dashboard
- POST /auth/register-profile - CHT Platform writes unified profile after GoTrue signup
- POST /auth/signup, /auth/login, Google OAuth - open to multiple user types
- PUBLIC_API_KEY on /api/public/* - correct pattern for CHT, but coexists with broad user JWT access

2.3 Why Change

MediaHub was designed as a unified identity hub. That made sense during prototyping but creates enterprise risk: pharma clients and clinicians should not authenticate into an internal ops tool; CHT should own the

clinician/KOL identity lifecycle via Cognito; MediaHub should expose data to CHT via API keys and receive admin operators via a separate Cognito app client restricted to CHM staff.

3. Target Identity Architecture (CHT-Owned Cognito)

3.1 Identity Ownership

AWS Cognito is the canonical IdP. CHT Platform team owns and operates the Cognito User Pool(s), app clients, hosted UI / custom login, MFA policy, and user lifecycle (signup, password reset, federated identity). This is a deliberate choice: Cognito is NOT being replaced or avoided; it becomes the sole human identity system for CHT end users and MediaHub admins (via separate app clients or groups).

3.2 Three Access Paths

Actor	Auth method	Access
CHT end user (KOL/HCP/pharma)	Cognito JWT via CHT app	CHT Platform only - NOT MediaHub UI
CHM admin/operator	Cognito JWT via MediaHub admin app client	MediaHub dashboard (admin/editor roles)
CHT backend / integrations	X-API-Key (PUBLIC_API_KEY)	/api/public/* read APIs
ops-console	X-API-Key (WEBHOOK_API_KEY)	/webhook/sync write APIs
Internal automation	IAM role / service token	Worker services, no user context

3.3 Cognito Configuration (Recommended)

1. User Pool: cht-platform-prod (owned by CHT AWS account or CHM sub-account with CHT admin access).
2. App client: cht-web - authorization code + PKCE for CHT Platform frontend (communityhealth.media).
3. App client: mediahub-admin - separate client; callback <https://mediahub.communityhealth.media/auth/callback>; restricted to Cognito group mediahub-admins.
4. Groups: mediahub-admins (superadmin/admin), mediahub-editors (editor), cht-kol, cht-hcp, cht-pharma-client - CHT groups never grant MediaHub access.
5. Custom claims: client_id, user_type, npa (optional) - embedded via pre-token generation Lambda for CHT; NOT forwarded to MediaHub unless admin.

6. API Gateway optional: Cognito authorizer in front of future public-facing CHT BFF; MediaHub /api/public stays API-key only.

3.4 MediaHub Auth Middleware Changes

- Replace GoTrue JWT verification with Cognito JWKS verification (python-jose or aws-jwt-verify)
- `get_current_user`: require Cognito group `mediahub-admins` OR `mediahub-editors` AND `public.users` row with role `admin/editor/superadmin`
- Reject all Cognito tokens from `cht-web` app client at MediaHub middleware (`audience/client_id` check)
- Remove: POST `/auth/signup`, public self-registration, CHT register-profile for non-service accounts
- Remove: GoTrue container from `docker-compose.prod.yml`
- Keep: invitation flow for CHM staff only - creates Cognito user in `mediahub-admin` group + `public.users` row
- Keep: `PUBLIC_API_KEY` and `WEBHOOK_API_KEY` - rotate and store in AWS Secrets Manager

3.5 GoTrue Decommission Plan

Step A: Provision Cognito + migrate admin users (Week 1-2)

- Create Cognito users for all active MediaHub admins/editors
- Add feature flag `AUTH_PROVIDER=cognito|gotrue` (dual-read during cutover)
- Update frontend login to Cognito Hosted UI / OIDC redirect

Step B: Lock down MediaHub to admin-only (Week 2-3)

- Remove `viewer/pharma_client` login paths from MediaHub frontend
- Deactivate or delete non-admin `public.users` rows (archive first)
- Return 403 for any Cognito token without `mediahub-admin/editor` group
- Notify CHT: use CHT Platform only; MediaHub URLs no longer for end users

Step C: Decommission GoTrue (Week 3-4)

- Remove gotrue service from docker-compose.prod.yml and staging
- Remove GOTRUE_* env vars, nginx /auth/v1 proxy, cors-gotrue.conf
- Remove GoTrueClient class and GoTrue JWT decode path from auth_service.py
- Drop auth schema tables if no other service depends on them (coordinate with chatbot)
- Update CHT register-profile endpoint to write only to public.users via internal API if needed for admin provisioning

4. Target Microservices Architecture

4.1 Service Decomposition

Service	Responsibility	Deploy
mediahub-api	HTTP API: analytics, content, admin, public API, webhooks	ECS Fargate (2+ tasks)
mediahub-worker	Cron sync, HCP intel poll, metric snapshots, tag jobs	ECS Fargate (1 task)
mediahub-render	FFmpeg GPU renders, timeline jobs	EC2 g4dn ASG or Batch
mediahub-reports	LLM report pipeline, PPTX assembly	ECS Fargate (scale 0-2)
mediahub-kb	Transcript correction, KB indexing queue	ECS Fargate (scale 0-2)
mediahub-web	Next.js admin frontend	ECS Fargate or Amplify
chm-chatbot	RAG search (existing, containerize)	ECS Fargate

4.2 Shared Infrastructure

- RDS PostgreSQL Multi-AZ - single database initially; schema-per-service later if needed
- ElastiCache Redis - job queues (replace BackgroundTasks), session cache, rate limit counters
- S3 - uploads, rendered videos, report PPTX, assets
- SQS - render jobs, report jobs, KB correction jobs, webhook retry DLQ
- Secrets Manager - API keys, Cognito config, LLM keys, OAuth tokens
- ALB - TLS termination, path routing: /api -> mediahub-api, / -> mediahub-web
- CloudWatch - logs, metrics, alarms; Sentry for application errors

4.3 Inter-Service Communication

Phase 1: All services share DATABASE_URL (modular monolith split, same repo). Phase 2: Workers consume SQS messages; API enqueues work and polls job status via Redis. Phase 3: Extract shared models into mediahub-common Python package; consider event bus (EventBridge) for post-sync and index-complete notifications.

4.4 What Stays in mediahub-api (Core API)

- routers/analytics.py, clients.py, tag_editor.py, public_api.py, webhook.py
- routers/auth.py (admin Cognito only, slimmed)
- routers/hcp_intel.py (read + admin; polling moves to worker)
- Synchronous CRUD; enqueue async work to SQS

4.5 What Moves Out

- services/scheduler.py -> mediahub-worker endpoint
- services/render_engine.py + routers/render.py job execution -> mediahub-render
- report_automation/ pipeline execution -> mediahub-reports
- kb_ai_corrector LLM passes + indexing -> mediahub-kb
- hcp_intel/orchestrator.py polling -> mediahub-worker

5. Codebase Migration Strategy

5.1 Repository Structure (Target)

Evolve chm-mediahub monorepo without fork. Introduce services/ layout:

```
chm-mediahub/  
  services/  
    api/          # FastAPI app (routers only)  
    worker/       # APScheduler + SQS consumers  
    render/       # FFmpeg worker  
    reports/      # Report pipeline worker  
    kb/           # KB correction worker  
  packages/  
    mediahub-core/ # models, config, database, shared services  
  frontend/       # Next.js (unchanged path)  
  infra/          # Terraform/CDK  
  docker/         # Per-service Dockerfiles
```

5.2 Strangler Fig Extraction Order

1. Extract mediahub-worker first - move start_scheduler() out of api lifespan; lowest user-facing risk.
2. Introduce SQS for render and report jobs - api enqueues, returns job_id; workers process.
3. Extract mediahub-render to GPU host - highest resource isolation win.
4. Extract mediahub-reports and mediahub-kb - LLM workloads isolated from API latency.
5. Split Docker images and ECS services - one task definition per service.
6. Optional: extract mediahub-core pip package for shared SQLAlchemy models.

5.3 Database Strategy

Keep single PostgreSQL database through Phase 2. All services use async SQLAlchemy with shared Alembic migrations in packages/mediahub-core/migrations. When splitting databases (Phase 3+), use bounded contexts: analytics schema, hcp_intel schema, kb schema - only after service APIs are stable. Never split DB

before worker extraction is proven.

5.4 API Contract Stability for CHT

CHT depends on /api/public/* endpoints. During migration:

- Maintain /api/public/* paths and response shapes - version with /api/public/v1/ if breaking changes needed
- PUBLIC_API_KEY rotation via Secrets Manager with dual-key grace period
- Publish OpenAPI spec from mediahub-api; CHT team signs off on changes
- No Cognito user token accepted on /api/public - API key only (already the case)
- Status endpoint GET /api/public/status remains unauthenticated for health polling

5.5 Frontend Changes

- Remove login/signup pages for non-admin user types
- Replace GoTrue token flow with Cognito OIDC (amazon-cognito-identity-js or oidc-client-ts)
- Admin-only route guard: redirect unauthenticated to Cognito login
- Remove client-facing dashboard views for pharma viewers OR move to CHT Platform
- CHATBOT_URL remains separate product; evaluate Cognito shared pool vs separate chatbot auth

6. Phased Migration Roadmap

Phase 0: Identity boundary + Cognito (parallel track) (Weeks 1-4)

- CHT provisions Cognito User Pool and app clients
- MediaHub admin app client + mediahub-admins Cognito group
- Dual-auth feature flag; migrate admin logins
- Lock MediaHub to admin-only; decommission GoTrue
- Rotate PUBLIC_API_KEY and WEBHOOK_API_KEY into Secrets Manager
- Document CHT integration contract (API key headers, endpoints, rate limits)

Phase 1: Infrastructure foundation (Weeks 3-8)

- RDS PostgreSQL Multi-AZ migration from Docker Postgres
- ElastiCache Redis migration
- S3 for uploads/reports/renders
- GitHub Actions CI/CD: lint, test, build, deploy to staging ECS
- ALB + ECS Fargate for mediahub-api + mediahub-web (staging first)

Phase 2: Worker extraction (Weeks 6-10)

- Extract mediahub-worker service; stop scheduler in API process
- SQS queues: render-jobs, report-jobs, kb-jobs
- API returns job IDs; poll /jobs/{id} for status
- Extract mediahub-render to GPU worker pool

Phase 3: LLM worker isolation (Weeks 10-14)

- Extract mediahub-reports and mediahub-kb containers

- Containerize chatbot; deploy to ECS on same VPC
- Remove NVIDIA runtime from mediahub-api container

Phase 4: Production cutover + hardening (Weeks 14-18)

- Production ECS cutover; decommission single-EC2 Docker Compose prod
- Enforce tenant isolation on analytics endpoints
- Fix LinkedIn metric mapping; analytics validation
- Monitoring, alerting, backup/restore runbooks
- Optional: Terraform IaC for full stack

6.1 Estimated Timeline

Total: 4-5 months with 1-2 engineers, running identity migration (Phase 0) in parallel with infrastructure (Phase 1). Phase 0 is a hard prerequisite before exposing MediaHub to any external stakeholder as an admin tool.

7. CHT Platform Integration Contract

7.1 What CHT Owns

- AWS Cognito User Pool - all end-user signup, login, MFA, password reset
- CHT Platform frontend (communityhealth.media, testapp.communityhealth.media)
- User profile data: NPI, user_type, Stripe Connect for KOL payments
- Clinician/KOL/pharma client UX - dashboards, social features, content discovery

7.2 What MediaHub Provides to CHT (API Key)

- GET /api/public/clips - clip catalog with engagement metrics
- GET /api/public/kols - KOL network data
- GET /api/public/playlists - playlist tags for carousels
- GET /api/public/status - health and latency for CHT monitoring
- POST /api/public/... (future) - only if write access needed; separate scoped keys

7.3 What MediaHub Does NOT Provide to CHT Users

- No MediaHub dashboard login for KOL/HCP/pharma users
- No shared GoTrue JWT acceptance for CHT app client
- No /auth/register-profile for CHT end-user provisioning into MediaHub
- No user invitation emails from MediaHub to external clients

7.4 Admin Access (CHM Staff Only)

CHM operators authenticate via Cognito mediahub-admin app client. Roles map from Cognito group + public.users.role: superadmin, admin, editor. Viewer and pharma_client roles are retired from MediaHub. Client-scoped data access for admins uses existing client_users table - enforced at API layer, not via CHT user tokens.

8. Risks, Decisions, and Open Questions

8.1 Key Risks

- Chatbot may still depend on GoTrue - must containerize and point to Cognito or separate pool before GoTrue removal
- Dual-auth cutover window - admins locked out if Cognito migration incomplete; require rollback plan
- CHT code may pass GoTrue JWT to MediaHub today - audit CHT codebase for token forwarding
- Worker extraction without SQS - do not split processes until queue is in place
- Single DB remains SPOF until RDS Multi-AZ - prioritize Phase 1 infra

8.2 Decisions Required from Leadership

1. Confirm Cognito User Pool lives in CHT AWS account vs CHM account with cross-account access.
2. Confirm all pharma client dashboard UX moves to CHT Platform (not MediaHub).
3. Approve GoTrue decommission date and chatbot auth migration plan.
4. Budget for RDS + ECS (~\$300-500/mo baseline vs current ~\$30 EC2).
5. Assign DRI for CHT side (Cognito) and CHM side (MediaHub admin + API).

8.3 Success Criteria

- Zero GoTrue containers in production
- Only Cognito admin-group users can access MediaHub UI
- CHT consumes all public data via API key only - verified by integration tests
- API and worker run as separate ECS services; API p95 latency stable under render load
- RDS automated backups with tested restore
- CI/CD deploys to staging on every main branch merge

9. Appendix

9.1 Files to Modify (Identity Migration)

- backend/services/auth_service.py - Cognito JWKS verify, remove GoTrueClient
- backend/middleware/auth.py - admin-only group check
- backend/routers/auth.py - remove public signup, slim to admin invite + Cognito callback
- backend/config.py - add cognito_user_pool_id, cognito_region, cognito_admin_client_id
- docker-compose.prod.yml - remove gotrue service
- nginx/mediahub.conf - remove /auth/v1 GoTrue proxy
- frontend/src/lib/auth-store.ts - Cognito OIDC token storage
- frontend/src/app/auth/* - replace GoTrue flows

9.2 Files to Modify (Microservices Extraction)

- backend/main.py - remove start_scheduler from lifespan
- backend/services/scheduler.py -> services/worker/main.py
- backend/routers/render.py - enqueue SQS instead of inline execute
- backend/routers/reports.py - enqueue SQS for pipeline
- backend/Dockerfile - split into docker/api.Dockerfile (no CUDA), docker/render.Dockerfile (CUDA)

9.3 Auth Flow Summary (Target)

CHT user opens communityhealth.media -> Cognito Hosted UI (cht-web client) -> JWT stored in CHT frontend -> CHT backend calls MediaHub GET /api/public/clips with X-API-Key header -> MediaHub returns data. CHT user never receives a MediaHub session.

CHM admin opens mediahub.communityhealth.media -> Cognito Hosted UI (mediahub-admin client) -> JWT with mediahub-admins group -> MediaHub API validates JWKS + group + public.users row -> admin dashboard access.

9.4 Related Documents

- CHM_MediaHub_Platform_Assessment_Report.pdf - current-state gaps and Needs Improvement ratings
- docs/WEBHOOK_API.md - ops-console webhook contract
- IMPLEMENTATION_PLAN.md - original architecture (requires update post-migration)